5 ANTICIPARE E GESTIRE GLI ERRORI

OBIETTIVI

- Spiegare tecniche per prevenire l'introduzione di errori nella tua soluzione.
- Discutere come mitigare gli effetti degli errori che vengono introdotti.
- Introdurre un approccio sistematico al testing, con strategie top-down e bottom-up.
- Mostrare come testare singole parti di una soluzione in isolamento.
- Discutere diverse strategie di debugging.
- Spiegare come gestire i bug trovati nella tua soluzione.

CONVIVERE CON I BUG

"Ricordo bene quando... mi resi conto con forza che avrei passato buona parte della mia vita a trovare errori nei miei programmi."
(Wilkes, 1985)

Devi accettare l'idea che le soluzioni che crei non funzioneranno perfettamente. Conterranno bug, nonostante tutti i tuoi sforzi. Anche i programmatori più talentuosi producono software con bug, anche se alcuni non lo ammettono volentieri.

Bug: un difetto in una soluzione che può causare un comportamento errato.

I bug possono essere scoperti e corretti in qualsiasi fase dello sviluppo, ma prima li trovi, meglio è. È molto più semplice correggere un errore quando è solo un'idea, piuttosto che quando è già stato implementato.

Errore: qualsiasi comportamento osservato durante l'esecuzione di una soluzione che non corrisponde a quello atteso.

Alcuni bug diventano evidenti solo dopo l'esecuzione della soluzione. Per questi casi, servono approcci come il **testing** e il **debugging**, che saranno trattati in questo capitolo.

PROGETTARE SENZA BUG

Questa sezione esamina i vari tipi di bug che possono causare comportamenti errati.

Errori di battitura (typo)

Gli errori più semplici da prevenire includono:

- errori di ortografia;
- lettere maiuscole errate;

- parole mancanti;
- parole sbagliate (es. *fase* invece di *frase*);
- numerazioni errate nelle liste.

Errori grammaticali e ambiguità

Anche una grammatica scorretta può creare problemi.

Esempio ambiguo:
"Dopo aver inserito una moneta, l'utente potrà spingere il tornello."
👉 **Chi ha inserito la moneta?** L'utente o il tornello?

Un altro esempio:
"Dopo alcuni tentativi di login falliti, l'incidente dovrebbe essere segnalato via email."
👉 **Cosa significa "alcuni tentativi"?** Due, cinque, cento?

💡 **Suggerimento**: mantieni un glossario dei termini chiave e usalo in modo coerente.

Incoerenze

Un'incoerenza si verifica quando parti della soluzione affermano cose che, messe insieme, non hanno senso.

Esempio:

1. *"Dopo 5 tentativi di login falliti, viene inviata un'email all'amministratore."*
2. *"Dopo 3 tentativi di login falliti, l'account viene bloccato."*

👉 **Contraddizione**: se l'utente viene bloccato dopo 3 tentativi, non potrà mai arrivare a 5!

Errori logici e matematici

Alcuni errori derivano da espressioni errate, anche se valide dal punto di vista sintattico.

Esempio errato (trova il problema!):

pseudocode

CopiaModifica

Se x > y, stampa y

Altrimenti, se y > x, stampa x

👉 **Errore**: stampa il valore minore invece del maggiore. Inoltre, cosa succede se x = y? Nulla viene stampato!

Altro esempio:

pseudocode

CopiaModifica

efficienza = miglia / galloni

👉 **Problema**: se galloni = 0, si genera un errore di divisione per zero!

💡 **Suggerimento**: i computer **non hanno buon senso**, quindi devi specificare ogni caso possibile.

MITIGARE GLI ERRORI

Nonostante tutti gli sforzi, la tua soluzione conterrà comunque bug. Puoi però **limitare i danni** adottando strategie difensive.

Programmazione difensiva

👉 *"Meglio prevenire che curare."*
L'idea è di controllare le condizioni prima di eseguire operazioni potenzialmente problematiche.

Esempio:

- Un ascensore, se perde potenza, attiva automaticamente i freni per sicurezza.
- Un software dovrebbe controllare che un numero non sia zero prima di usarlo come divisore.

Gestire gli errori

Se si verifica un errore, cosa dovrebbe fare il sistema?

Esempi:

- **Errore lieve** → mostrare un messaggio.
- **Errore critico** → spegnere il sistema per evitare danni.

💡 **Caso reale**:

- Un distributore automatico può restituire una moneta se non è leggibile.
- Se rileva una perdita d'acqua, potrebbe spegnersi per prevenire danni elettrici.

TESTING

Il **testing** serve a verificare se una soluzione funziona correttamente e a individuare i problemi.

Approcci al testing

1. **Top-down**: testare la soluzione nel suo insieme.
    - Vantaggi: aiuta a trovare errori di progettazione.
    - Svantaggi: difficile se il sistema non è completo.
2. **Bottom-up**: testare i componenti singolarmente.
    - Vantaggi: facilita l'individuazione di errori locali.
    - Svantaggi: gli errori di progettazione potrebbero emergere tardi.

👉 **Migliore strategia**: combinare entrambi!

💡 **Unit testing** → testare parti isolate della soluzione con dati di esempio.

DEBUGGING

Quando un errore si manifesta, come lo trovi?

Strategie di debugging

🔎 **Segui il metodo di Sherlock Holmes!**

1. **Ipotesi e verifica**
    - Se hai un sospetto, testalo. Se sbagli, scartalo subito.
2. **Rispetta il "Rasoio di Occam"**
    - La spiegazione più semplice è spesso quella giusta.
3. **Dividi e conquista**
    - Elimina intere parti della soluzione come "innocenti" per restringere la ricerca.
4. **Cambia una cosa alla volta**
    - Se cambi più cose insieme, non saprai quale ha risolto il problema.
5. **Usa il logging**
    - Aggiungi stampe di debug per vedere quali istruzioni vengono eseguite.
6. **Tracing manuale**
    - Simula l'esecuzione dell'algoritmo passo dopo passo.
7. **Spiega il problema ad alta voce**
    - Anche spiegandolo a qualcuno che non capisce, potresti avere un'illuminazione!

DECIDERE QUALI ERRORI RISOLVERE

Non puoi correggere tutto. Come scegli cosa sistemare?

1. **Gravità**:
    - 🔴 **Critico** → impedisce l'uso del sistema.
    - 🟡 **Moderato** → ostacola alcune funzioni.
    - 🟢 **Minore** → errore estetico (es. colore sbagliato).
2. **Priorità**:
    - 📌 **Frequenza** → quante volte accade?
    - 🔑 **Importanza** → coinvolge una funzione chiave?
    - ⏳ **Tempo di correzione** → vale la pena sistemarlo?